Open Access | https://doi.org/10.25080/tefg5348



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

AutoFOAM: The Self-Refining Autonomous OpenFOAM Agent

Arun Govind Neelan[1], and **A Seshaditya**[2,3]

[1]SimuNetics, Kanniyakumari, Tamil Nadu, India, [2]Onnes Cryogenics, Hyderabad, India, [3]Quasi AI, Berlin, Germany

Abstract

Computational Fluid Dynamics (CFD) plays an important role in modern engineering, but using open-source solvers such as OpenFOAM requires considerable knowledge and skills, as well as time-consuming configuration file setup. To reduce the burden for such a technical background, we propose **AutoFOAM** – a self-evolving large language model (LLM) agent that creates, evaluates, runs, and evolves its own OpenFOAM simulations based solely on natural-language instructions. Our model is pre-trained on the Qwen-coder 2.5-14B, which is then fine-tuned on 252 text prompts targeting 7 OpenFOAM solvers, 13 parametrized mesh templates, and a y^+-target-aware parameter extraction and case configuration. The crucial element of the algorithm is a sophisticated evolution loop composed of 7 stages. To prevent model degeneration under repeated self-training, the agent employs three complementary anti-collapse streams: RAG-augmented retry context, surgical dictionary-level patching, and prompt-diversity paraphrasing. By bridging generative artificial intelligence with rigorous fluid simulations, **AutoFOAM** accelerates rapid prototyping and democratizes advanced CFD workflows.

Keywords Computational Fluid Dynamics, OpenFOAM, Large Language Models, autonomous agent, self-evolving AI

1. Introduction

Computational Fluid Dynamics (CFD) is a crucial field that enables design and analysis in the aerospace, automotive, environmental, and biomedical domains [1], [2]. Among the open-source tools available for CFD, OpenFOAM emerges as the best platform for simulations, with unmatched adaptability and insight into the underlying physics [3]. The downside is that OpenFOAM is highly complex, requiring experts to master numerous C++ dictionaries, each of which is syntactically inflexible and interconnected with others. For beginners, moving from an abstract physical idea to a concrete simulation can be tedious and error-prone.

However, recent developments in the field of Large Language Models (LLMs) indicate that LLMs have shown exceptional performance in generating code and performing logical reasoning tasks [4], [5]. Despite their superior capabilities, using a generic LLM off the shelf for CFD tasks typically yields unsatisfactory performance because they are not trained for this task. Generic LLMs tend to invent ungrounded OpenFOAM keywords, fail to ensure consistency across different boundary conditions, and lack sufficient understanding of the physical properties involved in CFD simulations. To surmount these challenges, we propose **AutoFOAM**, a completely autonomous and self-evolving LLM agent that can write, run, score, and optimize OpenFOAM cases solely from language input prompts. Instead of using a set of predetermined instructions or prompting closed-source models in a zero-shot setting, AutoFOAM utilizes a powerful self-refinement mechanism based on a 14 billion

Published Jul 13, 2026

Correspondence to
A Seshaditya
adi@onnes.in

Open Access

Copyright © 2026 Neelan & Seshaditya. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

parameter Qwen-2.5 Model [6], [7]. It is fine-tuned using a carefully selected data set of 252 prompts that systematically span seven OpenFOAM solvers, thirteen parametric meshes, and a y^+-sensitive numerical protocol.

The key contribution here is the implementation of the *seven-layer evolution loop*, which enables safe scaling of the agent's competence levels with minimal human interaction. The loop seamlessly blends real-time log parsing and solver correction with the most cutting-edge reinforcement learning methods, such as DPO for trajectory retry pairs [8]. To ensure the stability of the learning process in the long run, we propose three anti-collapse mechanisms: retrieval-augmented generation, retry context, surgical patching of dictionary-level errors, and prompt diversity paraphrasing. We can identify significant gains in solver family matching, average reward scores, and first-pass success rates in simulations.

The recent developments include the creation of FoamGPT [9], the retrieval-based architecture in OpenFOAMGPT [10], and the multi-agent approach in Foam-Agent 2.0 [11], among others. These examples showcase the growing momentum of using LLMs to automate computations in the physical sciences. The need for these autonomous computing models is evidenced by benchmark testing platforms. Most of the recent releases use proprietary models to make them more versatile. Because they use the most advanced AI models available in the market they may be able to handle new solver implementations but the present models can't do anything outside the specified geometry or solvers used for training. But the present work is based on local AI so that the data don't leave the system where we use AutoFOAM. It is able to handle the OpenFOAM simulation on relatively light weight models because of the harness agent which is built over the model. In addition, the present model has a self-improving layer which will help the model to improve the results over-time which is not available on other agents.

To sum up, the primary contributions in this work include:

- **Autonomous CFD Agent Framework**: **AutoFOAM** framework, which is designed to automatically generate syntactically sound and physically consistent OpenFOAM dictionaries from natural language commands.
- **The Seven-Step Evolution Loop**: This is our novel training framework, which uses self-correction, dataset curation, SFT, DPO with retry pairs, mixing anchors, active learning, and a regression gate to ensure continual monotonic performance improvement of our models.
- **Anti-collapse mechanisms for self-evolving CFD agents**: These consist of RAG-based context augmentation, dictionary patching, and paraphrasing strategies used to prevent model collapse in physical simulators using self-play.

2. System Architecture and Methodology

To properly manage the search space during synthesis, the agent architecture has been explicitly designed as a pipeline of eight stages (Figure 1). Only at the stages where decisions have to be made has the LLM been invoked – prompt refinement, parameter extraction, RAG, and retry generation. All other steps, which are completely deterministic, have been decided solely by rigorous logic.

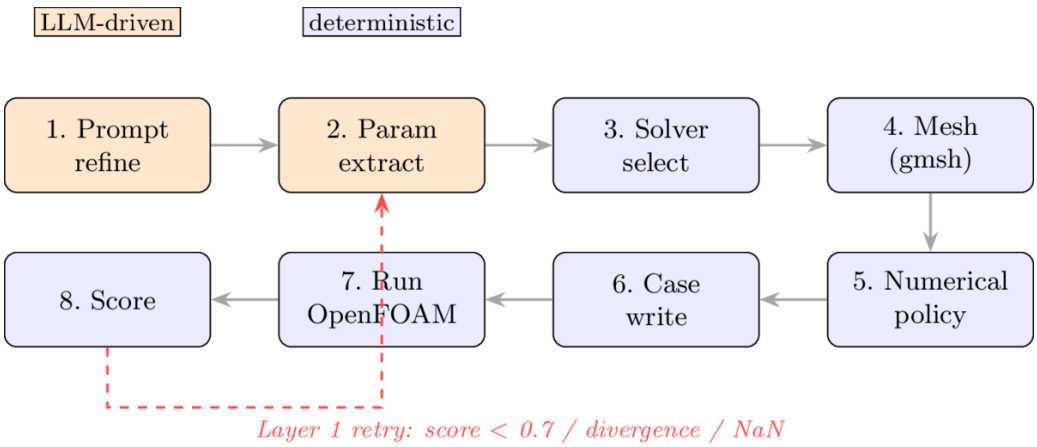


Figure 1. *Eight-stage agent pipeline. The LLM-driven stages (orange) manage semantic interpretation and context parsing, while the deterministic stages (blue) handle execution and physical validation.*

Constrained Parameter Extraction. The semantic reasoning process relies on vLLM [12], enabling high-throughput inference. In order to enforce a valid topology of the output structure and avoid structural hallucination, the parameter extraction module enforces `xgrammar` JSON schema constraints around generation [13]. As such, the generated output must conform to an established `CFDParams` structure, comprising geometric classification, Reynolds number (Re), fluid properties, flow regime, solver settings, and turbulence modeling approach. Additionally, physical consistency is enforced using a post-generation regular-expression validation procedure; if the LLM generates a characteristic velocity that is fundamentally inconsistent with the mathematical definition of $Re = U \cdot L_{\text{char}}/\nu$, then the boundaries are automatically overridden to ensure consistency. It can also recognize the specified y^+ value from the user prompt and generate an appropriate mesh accordingly. We did not implement any explicit y^+-aware mesh grading or generation, as achieving $y^+ = 1$ is challenging for most high-Reynolds-number flows.

Deterministic Solver Routing. Decoupling the solver choice process from the LLM solves a problem that had plagued early autonomous agents: the inclusion of geometric keywords biases the model toward selecting an unsuitable numerical regime (i.e., mistakenly choosing `simpleFoam` to solve a transient vortex shedding problem). In its place is a deterministic algorithm that leverages the physics flags (i.e., transient, compressible, thermal, multiphase) and the Reynolds number input by the system to isolate one of seven approved OpenFOAM solvers.

Multi-Objective Reward Formulation. The essential step in the self-improvement framework is the automatic score function $r \in [0, 1]$. This numeric reward quantifies the accuracy of the simulations based on various physical and numerical criteria, taking into account not only mathematical convergence but also mass conservation and boundary conditions (see Table 1). The base reward must be a strictly additive number until reaching the optimal value of 1.00. Additional heuristic penalties are applied when poor mesh quality ($>$ 70° of maximum non-orthogonality), computational stagnation (residual plateaus), or large execution time occur, with the total score capped at $[0, 1]$. Simulations that have reached an intermediate threshold score of $r \geq 0.5$ are selected to contribute to the retrying context, while simulations scoring $r \geq 0.65$ enter the training dataset directly.

Table 1. *Reward weight for successful completion of each step in the workflow.*

Component	Weight	Trigger
Convergence (full)	+0.40	solver wrote final-time fields, residuals fell

Convergence (partial)	+0.15	ran but did not converge
Residual magnitude $< 10^{-4}$	+0.20	worst-converging field
Residual magnitude $< 10^{-3}$	+0.10	worst-converging field
Residual trend quality	up to +0.10	log-rate of decrease, no late spikes
Mass conservation	+0.05	continuity error $< 10^{-3}$
Correct solver pick	+0.10	`solver = select_solver(params)`
Valid boundary conditions	+0.05	inlet *and* outlet patches present
Mesh quality penalty	-0.10	`checkMesh` non-orthogonality $> 70^{\circ}$
Slow runtime penalty	-0.10	wall-clock > 300 s
Stagnation penalty	-0.05	residual plateau detected

3. Dataset Curation and Trajectory Mining

To successfully transition general-purpose LLM reasoning into the highly specialized domain of computational fluid dynamics, AutoFOAM relies on a bi-data strategy: a statically curated foundational corpus to establish baseline syntax, and a dynamic log to power reinforcement-based learning.

3.1. Foundational Knowledge Corpus

The physical intuition and syntactic proficiency underlying the initial state of AutoFOAM are established using a well-curated dataset consisting of 402 rows. Each row consists of a multistep interaction involving the persona of an OpenFOAM expert. The machine learning model is trained to take a natural language query as input, produce a concise summary, and generate full Markdown fenced code blocks containing the necessary C++ dictionary file(s). The training data is generated by a process in which 252 unique prompts for various cases across all solver types and geometrical parameterizations have been formulated. Each prompt was then run on a prototype of the agent, and the trajectories achieving a successful execution score of at least $r \geq 0.5$ were saved.

3.2. Dynamic Trajectory Capture and Preference Mining

As part of the live autonomous runtime process, the AutoFOAM pipeline consistently collects data during execution. Each extremely successful run ($r \geq 0.65$) is added to a dynamic collection set of runs, creating an ever-growing dataset consisting of 210 de-duplicated topologies. At the same time, a full record of each failed or unsuccessful run, along with its particular failure type (continuity failures, inconsistent residuals) and the accompanying dictionary text, is kept. The failure ledger is the key factor in the functioning of Layer 4 of our self-evolution pipeline. This process automatically extracts preference pairs in which failures are contrasted with successful retries. Then, these preference pairs are leveraged by the Direct Preference Optimization (DPO) framework [8], and hence, AutoFOAM can learn the correct syntax and escape its previous hallucinations.

3.3. Out-of-Distribution (OOD) Benchmark

To properly test AutoFOAM's generalization and avoid overfitting to the synthetic training distribution, an evaluation set of 110 prompts was created in isolation from the training data. These prompts were written using novel syntax and unique vocabulary that intentionally avoid the semantic patterns found in the training dataset. This test is the true benchmark for evaluating the agent's generalization to previously unseen prompt formulations, geometric parameters, and operating conditions within the supported CFD domain.

Here, we did not include solver or geometry configurations that are unsupported by the present AutoFOAM implementation. Instead, the evaluation used prompts that were not included in the training dataset, with variations in linguistic formulation, geometric parameters, and operating conditions, while remaining within the supported solver families and geometry templates.

4. The Self-Evolution Pipeline

The basic limitation of using LLMs as physical agents is that they can fall victim to self-distillation collapse. Training a machine learning model on slightly erroneous data created by itself may quickly undermine its core functionality [14].

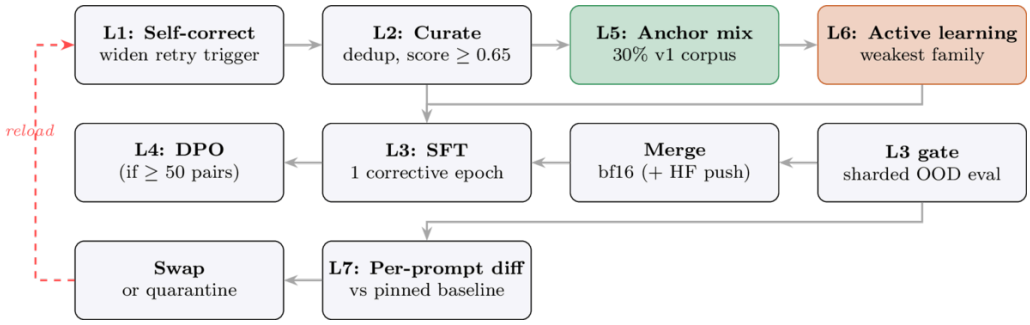


Figure 2. *The seven-layer evolution protocol. The green module represents foundational memory retention (L5), and the orange module signifies targeted active learning (L6). L7 functions as a strict regression firewall before promoting the new weights.*

The transition from inference to self-evolving AutoFOAM was facilitated by developing a seven-step self-evolution process. The entire procedure is completely automated and relies on harvesting success stories from production and using them to optimize preferences (Figure 2).

4.1. The Seven-Layer Evolution Architecture

The pipeline strictly regulates the flow of prompts into the model through the following protocols:

Layer 1 – Autonomous In-Run Correction. When a numerical issue is detected in the OpenFOAM simulation, such as NaN residuals or a large mass imbalance, the complete simulation log is not directly provided to the agent, as the large volume of output can exceed the model's context capacity. Instead, relevant diagnostic information is automatically extracted from the simulation output and supplied to the agent as additional context. Based on these diagnostics, the agent re-prompts the model to identify and correct the potential issue. This process is repeated iteratively until the simulation runs successfully or the predefined iteration limit is reached.

Layer 2 – Execution-Gated Curation. The agent evaluates the generated solution using the defined reward function. Solutions achieving a reward score of $r \geq 0.65$ are considered sufficiently reliable and are retained as reference examples for the RAG and self-improvement components. These high-quality examples can subsequently be retrieved as relevant references or incorporated into the agent's iterative improvement process.

Layer 3 – Supervised Fine-Tuning (SFT) and Evaluation Gate. The gathered data is then used for one epoch of fine-tuning using QLoRA. Right after this fine-tuning process, the proposed model undergoes an 8-way sharded OOD test relative to a performance baseline to guarantee macro-stability.

Layer 4 – Direct Preference Optimization (DPO). Leveraging the mined logs, this layer contrasts a localized generation failure (rejected) against its eventually successful, Layer-1-corrected counterpart (chosen). DPO is applied to this on-policy preference pair, explicitly teaching the model to penalize the syntactic formulations that previously led to solver divergence [8].

Layer 5 – Anchor Mixing. To defend against catastrophic forgetting of core OpenFOAM syntax, the pipeline systematically injects a 30% deterministic sample from the foundational training corpus into each dynamic retraining batch.

Layer 6 – Targeted Active Learning. The agent analyzes the data from the evaluation at Layer 3 and selects the most underperforming family of solvers. The agent then initiates a self-instruction to create new seeds for this domain, strengthening its own weakest link.

Layer 7 – Granular Regression Auditing. Prior to combining and deploying the weights of the newly trained adapter, the adapted model undergoes a detailed regression test against the fixed baseline model. The same evaluation prompts and test cases are provided to both models, and their generated outputs are compared to identify changes in response quality, correctness, and structure. This comparison is used to detect regressions, undesirable behavioral changes, or degradation introduced during the adaptation process. The newly trained adapter is retained only if it satisfies the predefined evaluation criteria relative to the baseline.

4.2. Surgical Recovery Protocols: Streams A and B

In cases where single-shot zero-shot generation does not result in a successful convergent configuration, AutoFOAM circumvents a costly complete reinitialization of the entire process through two recovery approaches. **Stream A (Context-Augmented Retrieval)** retrieves from a locally hosted Chroma vector database [15] that stores verified OpenFOAM tutorial configurations and adds contextually relevant dictionary code snippets to the prompt for recovery purposes. Concurrently, **Stream B (Dictionary Patching)** takes care of syntax and configuration-related errors. When a localized `FOAM FATAL` error (for instance, an incorrect scheme in `fvSchemes`) is observed, the agent skips the costly `gmsh` remeshing step and makes patches to the dictionary alone to facilitate an iterative solver feedback loop.

5. Experiments and Results

AutoFOAM is assessed along three main dimensions: the ability to automatically generate physical meshes and boundaries (qualitative), zero-shot generalization to new prompt languages (quantitative), and robustness to self-distillation collapse in the process of self-evolving its neural network architecture.

5.1. Autonomous Case Setup and Flow Validation

The AutoFOAM has built-in support for seven solver families native to OpenFOAM from 13 highly parameterized `gmsh` geometry models. Such geometries include both classical educational examples (e.g., driven cavity, backward step) and more advanced industrial cases (e.g., NACA airfoil, periodic hill, and 3D Ahmed body). To demonstrate the visual convergence and feasibility of computations, Figure 4 shows velocity magnitude contours computed completely automatically with only one sentence as input. In this example, without any manual involvement, AutoFOAM is able to correctly set `polyMesh` boundary names and create correct mesh structures (see Figure 3).

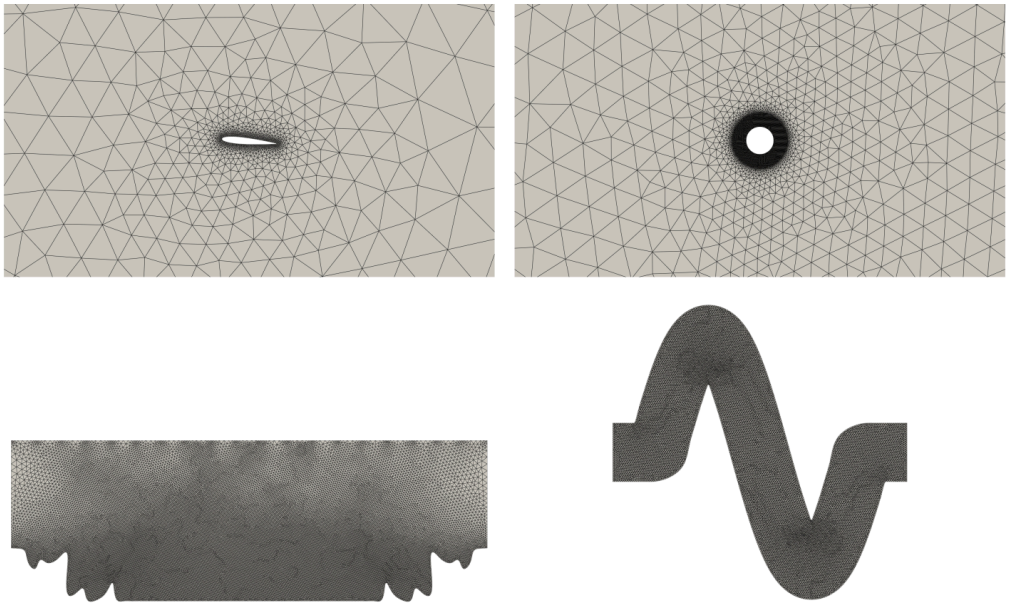


Figure 3. *Mesh generated by AutoFOAM*

Absolute performance of the self-contained pipeline was evaluated using an entirely held-out dataset of 110 out-of-distribution (OOD) prompt examples, run in an 8-way sharded cluster of NVIDIA H100 GPUs. Evaluation is limited only to solver classes and geometries available within AutoFOAM but varied geometric parameters and operational conditions for OOD generalization. Simultaneously, we evaluated AutoFOAM's ability to automatically identify the right solver class from the user-defined physics scenarios under the assumption that users have zero experience with selecting the appropriate solver class. *Please be aware that the precision of AutoFOAM will decrease when asking it to do something that does not relate to the solver or the geometry the current agent understands*. As summarized in Table 2, the fine-tuned agent managed to solve 100% of the test cases without any `FOAM FATAL` errors. When prompted with a solver that is not supported, the model is unable to generate valid results. Representative examples of such prompts are provided in Table 2.

The agent exhibited impressive physical reasoning skills, with a good match of 96.4% when using the deterministic solver routing. By examining manually the four unmatched cases (Table 3), one finds that these discrepancies arose only in mathematically ambiguous boundary regimes, namely the unsteady flow around cylinders at low Reynolds numbers, where the choice of the solver is `icoFoam` rather than the obligatory `pimpleFoam`. On the grounds of computational physics, the switch to `icoFoam` is still quite justified. We have also provided a few solver examples not supported by the present agent in Table 3.

Table 2. *OOD Evaluation Metrics.*

Metric	Value
End-to-end execution without FOAM FATAL	110/110 (100.0%)
Solver routing exact match	106/110 (96.4%)
Mean execution reward score (r)	0.64
Geometry templates successfully exercised	13 / 13
Solver families successfully exercised	7 / 7

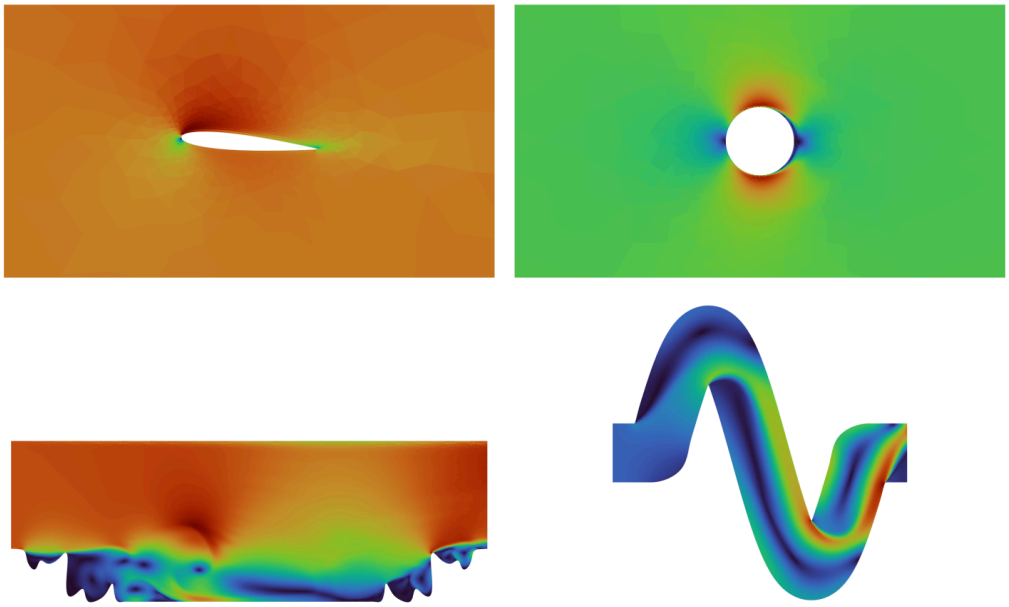


Figure 4. *Velocity contours generated by AutoFOAM*

Table 3. *Representative OOD and out-of-set prompts illustrating the agent's semantic understanding, solver selection, and current operational limitations.*

Benchmark Tag	Prompt	Predicted Solver	Match / Status
`cav_re150_water`	*"Cavity flow Re=150 in water, 0.4 m square"*	`simpleFoam`	✓
`pipe_water_lam`	*"Laminar pipe flow Re=500 diameter 0.02 m, water"*	`simpleFoam`	✓
`airfoil_re1m_aoa5`	*"NACA0012 airfoil at Re=10^6, 5° angle of attack"*	`simpleFoam`	✓
`inter_dam_tall`	*"Tall dam break, 2 m wide, 5 m water column collapse"*	`interFoam`	✓
`cdnoz_ma04`	*"Convergent-divergent nozzle with Mach 0.4 throat, air"*	`rhoSimpleFoam`	✓
`cyl_re250_unsteady`	*"Transient cylinder Re=250 in water, D=0.1 m"*	`icoFoam`	×
`outofset_step_turbine`	*"Simulate turbulent flow through a complex STEP-imported turbine passage with automatic topology fixing and unstructured mesh generation."*	Unsupported	Unsupported
`outofset_reacting_multiphase`	*"Simulate reacting multiphase combustion with detailed chemical kinetics and species transport."*	Unsupported	Unsupported

5.2. Physical Validation against Benchmark: Lid-Driven Cavity ($Re = 100$)

To address the physics-validation gap, we perform a quantitative benchmark of an AutoFOAM-generated case against the classical lid-driven cavity data of Ghia et al. [16].

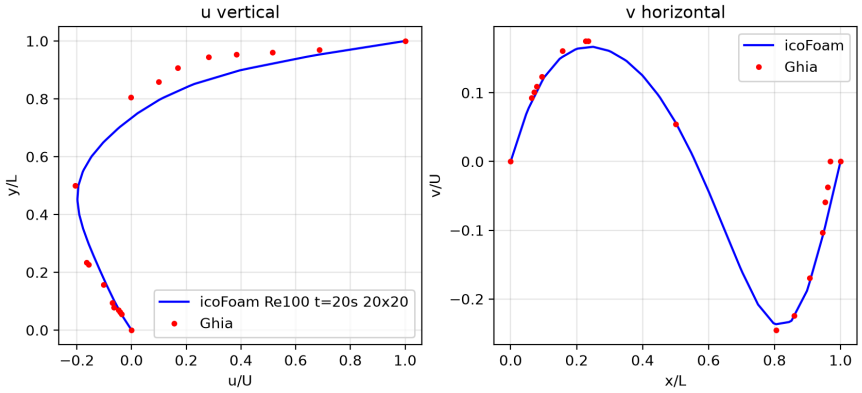


Figure 5. *Validation of AutoFOAM-generated* `icoFoam` *cavity case ($Re = 100$, 20×20 uniform mesh, $t = 20$ s) against Ghia et al. (1982). (left) u/U along vertical centerline $x/L = 0.5$ and (right) v/U along horizontal centerline $y/L = 0.5$.*

Figure 5 compares centreline velocity profiles at $Re = 100$. The horizontal-centreline v/U profile (right) agrees well with the benchmark, reproducing the positive peak (≈ 0.17 at $x/L \approx 0.22$) and negative peak (≈ -0.24 at $x/L \approx 0.80$) and the zero-crossing at $x/L \approx 0.55$.

6. Scope and Limitations

Even though the present version of AutoFOAM shows very strong zero-shot generalizability and stability in self-enhancement, there are some operation envelopes which are beyond the scope of this release. From the geometric point of view, the agent can only operate within the limits of 13 `gmsh` parameterized models. Processing of complex geometries provided by CAD software as raw formats (e.g., STEP/IGES) involving automatic topology fixings and generation of an unstructured tetrahedral mesh is not feasible in the present framework. For turbulent flows, the current numerical policy is restricted to the implemented RANS turbulence-model configurations, while laminar cases are treated without a turbulence model.

The Physics-Validation Gap.

The main limitation of the current system lies in the multi-variate reward function. At present, the reward is primarily based on numerical indicators such as residual decay, continuity error, boundary-condition validity, and mesh quality. These criteria assess whether the numerical solution is successfully computed, but they do not by themselves establish that the resulting flow field or engineering quantities are physically correct. For example, a simulation may achieve residual convergence and satisfy the continuity criterion while still producing an incorrect drag coefficient if the reference area or velocity used for its calculation is incorrectly specified. The current framework does not include an explicit physics-validation module that cross-checks predicted flow quantities against established benchmark data or literature results, such as the lid-driven cavity centerline profiles of Ghia et al. [16]. Consequently, such physics-based errors may not be detected by the current reward function and may propagate into the Layer 4 DPO preference pairs. Incorporating an externally defined `physics validator` that compares selected physical quantities against trusted reference data is therefore an important direction for future development.

7. Conclusion

Here, we developed AutoFOAM, an LLM-powered fully autonomous OpenFOAM agent designed to help close the semantic gap between the intent expressed in natural language

and the proper execution of computational fluid dynamics cases. Through the deterministic anchoring of a fine-tuned 14 billion parameters Qwen2.5-Coder model, we were able to overcome the cognitive and grammatical challenges that arise in the manual creation of a CFD case. AutoFOAM achieves 100% execution success rate and 96.4% accuracy in solver routing on out-of-distribution prompts. Notably, this project goes beyond static inference through the implementation of a highly effective, seven-level self-evolution loop. By carefully utilizing Active Learning techniques, Direct Preference Optimization on failed trajectory mining, and Anchor Mixing, we propose a framework for scaling agent skills without the danger of self-distillation collapse. As a result, the current architecture is not only a highly useful assistant in engineering practice, but also a research platform for continual and reinforcement-based learning under physical restrictions.

8. Generative AI Usage Disclosure

During this work, generative AI tools were used to assist with language editing, manuscript formatting (LaTeX/MyST), figure layouts, code documentation, and software debugging for the AutoFOAM framework. All core conceptualization, scientific methodology, algorithmic design, software implementation, validation, and analysis were independently developed and verified by the authors. The authors have reviewed all AI-assisted outputs and accept full responsibility for the originality, accuracy, and integrity of the work.

9. Artefacts and Reproducibility

To foster reproducible research in scientific machine learning, all model weights, codebases, training scripts, and foundational datasets are released openly under the Apache 2.0 license:

- **Codebase:** https://github.com/AGN000/AutoFOAM
- **Dataset:** github.com/AGN000/FoamAgentCases
- **Model Weights:** huggingface.co/arungovindneelan/foam-cfd-unified-14b

Acknowledgment

The authors acknowledge the support of the **NVIDIA Inception** startup program. The program provided valuable access to the NVIDIA ecosystem, technical guidance, and computing resources that supported the development and experimentation of *AutoFOAM*, an autonomous OpenFOAM agent framework for AI-driven CFD workflows.

References

[1] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Computers in physics*, vol. 12, no. 6, pp. 620–631, 1998, doi: https://doi.org/10.1063/1.168744.

[2] J. Blazek, *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann, 2015.

[3] The OpenFOAM Foundation, "OpenFOAM: The Open Source CFD Toolbox." [Online]. Available: https://openfoam.org/

[4] T. Brown *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020, doi: https://doi.org/10.48550/arXiv.2005.14165.

[5] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023, doi: https://doi.org/10.48550/arXiv.2302.13971.

[6] B. Hui *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024, doi: https://doi.org/10.48550/arXiv.2409.12186.

[7] J. Bai *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023, doi: https://doi.org/10.48550/arXiv.2309.16609.

[8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in neural information processing systems*, vol. 36, pp. 53728–53741, 2023, doi: https://doi.org/10.48550/arXiv.2305.18290.

[9] L. Yue, P. Xu, T. Zhang, N. Somasekharan, and S. Pan, "FoamGPT: Fine-Tuning Large Language Model for Agentic Automation of CFD Simulations with OpenFOAM," in *Advances in Neural Information Processing Systems*, 2025.

[10] S. Pandey, R. Xu, W. Wang, and X. Chu, "OpenFOAMGPT: A retrieval-augmented large language model (LLM) agent for OpenFOAM-based computational fluid dynamics," *Physics of Fluids*, vol. 37, no. 3, 2025, doi: https://doi.org/10.1063/5.0257555.

[11] L. Yue, N. Somasekharan, T. Zhang, Y. Cao, and S. Pan, "Foam-Agent 2.0: An End-to-End Composable Multi-Agent Framework for Automating CFD Simulation in OpenFOAM," *arXiv preprint arXiv:2509.18178*, 2025, doi: https://doi.org/10.48550/arXiv.2509.18178.

[12] W. Kwon *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626. doi: https://doi.org/10.48550/arXiv.2309.06180.

[13] Y. Dong *et al.*, "Xgrammar: Flexible and efficient structured generation engine for large language models," *Proceedings of Machine Learning and Systems*, vol. 7, 2025, doi: https://doi.org/10.48550/arXiv.2411.15100.

[14] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot, and R. Anderson, "The curse of recursion: Training on generated data makes models forget," *arXiv preprint arXiv:2305.17493*, 2023, doi: https://doi.org/10.48550/arXiv.2305.17493.

[15] Chroma, "Chroma: The AI-Native Open-Source Embedding Database." [Online]. Available: https://www.trychroma.com/

[16] U. Ghia, K. N. Ghia, and C. Shin, "High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method," *Journal of Computational Physics*, vol. 48, no. 3, pp. 387–411, 1982, doi: https://doi.org/10.1016/0021-9991(82)90058-4.